Integrated Symbolic Planning and Robotic Execution: A PDDL-Based Task Planner with ROS 2 and Gazebo

Andrew Khalil Abdelmalak
Dept. of Mechatronics Engineering
German University in Cairo (GUC)
Cairo, Egypt

Daniel George Boules
Dept. of Mechatronics Engineering
German University in Cairo (GUC)
Cairo, Egypt

David Louis Girgis
Dept. of Mechatronics Engineering
German University in Cairo (GUC)
Cairo, Egypt

Samir Sameh Yacoub
Dept. of Mechatronics Engineering
German University in Cairo (GUC)
Cairo, Egypt

Youssef Youssry
Dept. of Mechatronics Engineering
German University in Cairo (GUC)
Cairo, Egypt

***Abstract*—This paper presents the design and implementation of an integrated symbolic planning and robotic execution system for autonomous mobile manipulation. A PDDL-based task planner using Fast Downward with A* search and the LM-Cut heuristic generates action plans from high-level goals, which are then translated into physical robot commands through a modular ROS 2 architecture. The system operates a TurtleBot3 with an OpenManipulator arm in the Gazebo simulation environment, performing autonomous navigation via Nav2 and object manipulation through sequential gripper control. A Knowledge Base (KB) is updated in closed-loop fashion only after successful physical execution, ensuring state consistency between the symbolic and physical layers. Experimental results demonstrate successful end-to-end task completion, including multi-room navigation with 0.30 m tolerance and pick-and-place manipulation sequences.**

***Index Terms*—Task planning, PDDL, ROS 2, mobile manipulation, TurtleBot3, Gazebo, Nav2, symbolic AI, autonomous robotics.**

I. Introduction

Autonomous robotic systems that can reason about high-level goals and translate them into low-level physical actions represent a central challenge in AI for robotics [1]. While classical robotics excels at motion planning and control, bridging the gap between symbolic reasoning and continuous robot execution remains an active research area [2].

This work addresses the integration challenge by combining three key capabilities:

- **Symbolic Planning:** A PDDL-based planner (Fast Downward [3]) generates action sequences from declarative goal specifications.
- **Plan Translation:** A bridge layer maps symbolic actions (e.g., `move kitchen`) to continuous ROS 2 commands with appropriate coordinates and tolerances.
- **Physical Execution:** The TurtleBot3 with OpenManipulator autonomously navigates between rooms using Nav2 and manipulates objects using sequential gripper control.

A critical design decision is the closed-loop Knowledge Base (KB) update: the symbolic state is modified only after confirmed physical execution, preventing plan-reality divergence [4].

II. System Architecture

A. World Model

The environment models a multi-room apartment with six rooms, each assigned navigation coordinates within the Gazebo House World (Table I).

TABLE I
Room Coordinates in Gazebo World Frame

Room	x (m)	y (m)
Storage	−4.5	−1.5
Living Room	−1.0	1.5
Dining Room	6.0	1.0
Bathroom	3.0	5.0
Bedroom	−4.5	3.5
Kitchen	8.5	−1.5

Ten objects are distributed across rooms (e.g., cup in kitchen, book in bedroom) and tracked in a Python KB that synchronizes with the PDDL problem definition.

B. PDDL Domain Specification

The symbolic layer uses the following predicates: `robot-at(r)`, `at(o,r)`, `handempty()`, `holding(o)`, and `connected(r1,r2)`. Three action schemas define the planning operators:

- **`Move(from, to)`**: Preconditions: `robot-at(from)`, `connected(from,to)`. Effects: `robot-at(to)`, ¬`robot-at(from)`.
- **`Pick(o, r)`**: Preconditions: `robot-at(r)`, `at(o,r)`, `handempty`. Effects: `holding(o)`, ¬`at(o,r)`, ¬`handempty`.

- **`Place(o, r)`**: Preconditions: `robot-at(r)`, `holding(o)`. Effects: `at(o,r)`, `¬holding(o)`, `handempty`.

C. Planning Engine

The planner uses Fast Downward [3] with A* search and the LM-Cut heuristic, producing optimal action sequences. Fig. 1 shows the CLI interface with the initial KB state, while Fig. 2 demonstrates a complete plan for the goal `at cup storage`.

```
daniel-boules@danielboules-ASUS:~/Semester9/AI/Milestone2$ python3 src/main.py

--- Current KB ---
Robot at: livingroom
Objects: {'cup': 'kitchen', 'pan': 'kitchen', 'plate': 'diningroom', 'notebook': 'diningroom',
 'bottle': 'livingroom', 'remote': 'livingroom', 'book': 'bedroom', 'phone': 'bedroom', 'keys'
: 'storage', 'towel': 'bathroom'}
Holding: None

Enter goal (or 'quit'):
```

Fig. 1. CLI interface showing the initial Knowledge Base state: robot in the living room, ten objects distributed across six rooms.

```
Enter goal (or 'quit'): at cup storage

Plan found:
['move', 'livingroom', 'diningroom']
['move', 'diningroom', 'kitchen']
['pick', 'cup', 'kitchen']
['move', 'kitchen', 'diningroom']
['move', 'diningroom', 'livingroom']
['move', 'livingroom', 'bedroom']
['move', 'bedroom', 'storage']
['place', 'cup', 'storage']

--- Current KB ---
Robot at: storage
Objects: {'pan': 'kitchen', 'plate': 'diningroom', 'notebook': 'diningroom', 'bottle': 'living
room', 'remote': 'livingroom', 'book': 'bedroom', 'phone': 'bedroom', 'keys': 'storage', 'towe
l': 'bathroom', 'cup': 'storage'}
Holding: None
```

Fig. 2. Plan execution for goal "at cup storage": the planner generates an 8-step sequence (six `move`, one `pick`, and one `place`) traversing five rooms. The KB updates after each confirmed action.

III. ROS 2 Integration

A. Architecture Overview

The execution pipeline consists of three layers:

1) **Plan Executor** (`executor.py`): Parses PDDL actions and launches appropriate ROS 2 nodes via Python subprocesses.
2) **Navigation Node**: Interfaces with the Nav2 stack, sending target coordinates as `/navigate_to_pose` action goals with a configurable tolerance of 0.30 m.
3) **Gripper Node**: Controls the OpenManipulator via `/arm_controller/follow_joint_trajectory` and `/gripper_controller/gripper_cmd`, implementing a three-phase manipulation sequence.

Table II summarizes the key ROS 2 interfaces.

TABLE II
KEY ROS 2 INTERFACES

Component	Type	Topic/Action
Navigation	Action Client	`/navigate_to_pose`
Arm Control	Action Client	`/arm_controller/...`
Gripper	Action Client	`/gripper_controller/...`

B. Launch Configuration

Two launch files orchestrate the simulation:

- **Gazebo Launch**: Loads the apartment world (`my_house.world` with the custom `tb3_house_custom` model based on `turtlebot3_house.world`), spawns the robot model at configurable coordinates, and publishes TF transforms.
- **Navigation Launch**: Brings up the Nav2 stack [5] (AMCL localizer, global/local planners), loads the custom SLAM occupancy grid (`map.yaml`), and starts RViz2 for visualization.

IV. Experimental Results

A. Navigation Performance

The robot successfully navigated between all room pairs using Nav2's global planner (Fig. 3). The 0.30 m positional tolerance prevents oscillatory behavior near goal positions while maintaining sufficient accuracy for the symbolic room-level abstraction.

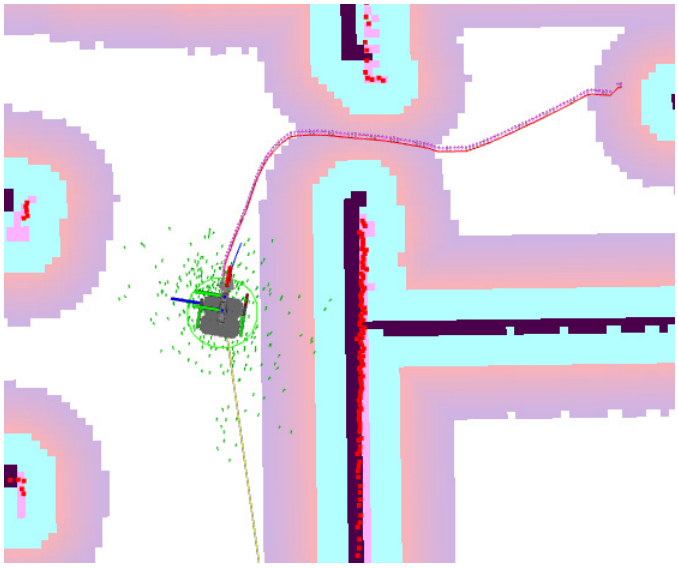
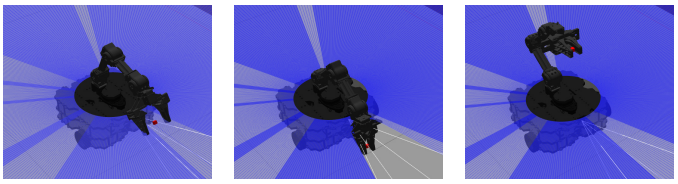


Fig. 3. Navigation execution in RViz: the global planner (red line) generates a path from the Dining Room to the Kitchen, overlaid on the costmap. The robot (center) with LIDAR scan (green) successfully follows the planned trajectory.

B. Manipulation Sequence

Object manipulation follows a three-phase sequence validated in Gazebo (Fig. 4):

1) **Approach:** Arm lowers to pre-set pick/place coordinates.
2) **Actuate:** Gripper opens (place) or closes (pick) based on the PDDL action.
3) **Retract:** Arm returns to the safe driving home pose.



(a) Arm down (b) Actuate (c) Home pose

Fig. 4. Three-phase manipulation sequence in Gazebo: (a) arm lowers to approach position, (b) gripper actuates (open/close), (c) arm returns to safe home pose for driving.

C. End-to-End Task Completion

We tested the integrated system with the goal `at cup storage`, requiring:

- Navigation across 5 rooms via 6 move actions (livingroom → diningroom → kitchen → diningroom → livingroom → bedroom → storage).
- One pick action in the kitchen and one place action in storage.
- KB updated only after each physical action succeeds.

V. Discussion

The closed-loop KB update strategy proved essential for system reliability. By deferring symbolic state changes until physical execution confirms success, the system avoids the "plan divergence" problem common in open-loop execution architectures [4].

The 0.30 m navigation tolerance represents a pragmatic trade-off: tighter tolerances caused oscillation near goals, while looser tolerances risked the robot stopping outside rooms. The room-level symbolic abstraction means exact positioning is less critical than reliable arrival.

Limitations include the absence of visual object detection (objects are assumed at known positions) and the lack of replanning capability on execution failure. Integrating computer vision for object localization and failure recovery would significantly enhance system robustness.

VI. Conclusion

We demonstrated a complete pipeline from symbolic AI planning to physical robotic execution. By connecting a Fast Downward PDDL planner to the ROS 2 ecosystem via a modular execution bridge, the TurtleBot3 with OpenManipulator autonomously navigates a multi-room environment and performs pick-and-place tasks based on high-level goal specifications. The closed-loop KB architecture ensures state consistency between symbolic reasoning and physical reality.

References

[1] M. Ghallab, D. Nau, and P. Traverso, *Automated Planning: Theory and Practice*. Morgan Kaufmann, 2004.

[2] L. P. Kaelbling and T. Lozano-Pérez, "Hierarchical task and motion planning in the now," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2011, pp. 1470–1477.

[3] M. Helmert, "The Fast Downward planning system," *J. Artificial Intelligence Research*, vol. 26, pp. 191–246, 2006.

[4] F. Ingrand and M. Ghallab, "Deliberation for autonomous robots: A survey," *Artificial Intelligence*, vol. 247, pp. 10–44, 2017.

[5] S. Macenski, F. Martín, R. White, and J. G. Crespo, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2020.